

---

# Arquitectura de despliegue

Portal institucional

Agencia ITRC

---

Documento técnico de referencia

Dirigido a equipo de infraestructura y administradores del sitio

## Contenido del documento:

- Migración de WordPress a JAMstack (Astro + Sveltia CMS)
- Despliegue automatizado con *runner* self-hosted en el servidor
- Arquitectura sobre servidor institucional (Ubuntu + nginx)
- Flujo de trabajo Git con múltiples webmasters
- Manejo separado de contenido (Git) y binarios (servidor)
- Política de respaldos local (snapshots con *hardlinks*)

# Índice

---

|                                                                                           |           |
|-------------------------------------------------------------------------------------------|-----------|
| <b>1. Resumen ejecutivo</b>                                                               | <b>2</b>  |
| 1.1. Componentes principales . . . . .                                                    | 2         |
| <b>2. Diagrama de arquitectura</b>                                                        | <b>3</b>  |
| <b>3. Flujo de publicación paso a paso</b>                                                | <b>4</b>  |
| 3.1. Escenario 1: Webmaster edita con VS Code (flujo principal hoy) . . . . .             | 4         |
| 3.2. Escenario 2: Webmaster edita con Sveltia CMS ( <i>pendiente</i> ) . . . . .          | 4         |
| 3.3. Escenario 3: Subida de binarios al servidor . . . . .                                | 5         |
| 3.4. Escenario 4: <i>Deploy</i> manual de fallback . . . . .                              | 5         |
| <b>4. Configuración del servidor (Ubuntu + nginx)</b>                                     | <b>6</b>  |
| 4.1. Servidor de pruebas actual . . . . .                                                 | 6         |
| 4.2. Cuentas y permisos del servidor . . . . .                                            | 6         |
| 4.3. Estructura del sitio en el servidor . . . . .                                        | 6         |
| 4.4. Configuración nginx mínima (estado actual) . . . . .                                 | 6         |
| <b>5. Autenticación y administración de usuarios</b>                                      | <b>8</b>  |
| 5.1. Modelo de cuentas . . . . .                                                          | 8         |
| 5.2. Flujo de autenticación (OAuth con GitHub) — <i>pendiente de despliegue</i> . . . . . | 8         |
| 5.3. Alta de un webmaster nuevo . . . . .                                                 | 8         |
| 5.4. Roles sugeridos . . . . .                                                            | 9         |
| 5.5. Baja de un webmaster . . . . .                                                       | 9         |
| 5.6. Buenas prácticas de seguridad . . . . .                                              | 9         |
| <b>6. Consideraciones operativas</b>                                                      | <b>10</b> |
| 6.1. Manejo de binarios . . . . .                                                         | 10        |
| 6.2. Rollback (revertir un cambio) . . . . .                                              | 10        |
| 6.3. Monitoreo y logs . . . . .                                                           | 11        |
| 6.4. Política de respaldos local . . . . .                                                | 11        |
| 6.5. Respaldo del repositorio (Git) . . . . .                                             | 11        |
| <b>Referencias</b>                                                                        | <b>12</b> |

# 1 Resumen ejecutivo

El portal institucional de la Agencia ITRC migró desde una arquitectura WordPress (dinámica, basada en base de datos MySQL y PHP) hacia una arquitectura **JAMstack** basada en el generador de sitios estáticos [Astro](#). El portal se despliega sobre el servidor institucional del ITRC (Ubuntu con nginx) mediante un *runner* self-hosted de GitHub Actions que vive en el mismo servidor.

## Cambios clave respecto al modelo anterior

- **Sin base de datos:** todo el contenido editorial es archivos JSON y Markdown versionados en Git.
- **Sin PHP:** el servidor únicamente sirve archivos estáticos (HTML, CSS, JS, PDF, imágenes).
- **Git como fuente de verdad para contenido:** cada cambio es un *commit* con trazabilidad y posibilidad de *rollback*.
- **Despliegue automatizado:** cada `git push` a `main` dispara un workflow que construye y publica el sitio sin intervención manual.
- **Runner self-hosted:** la construcción ocurre dentro del propio servidor, evitando exponer SSH a la nube y eliminando llaves remotas.
- **Binarios fuera del repo:** PDFs y multimedia residen sólo en el servidor, con respaldo local nocturno (decisión revisada en mayo 2026).

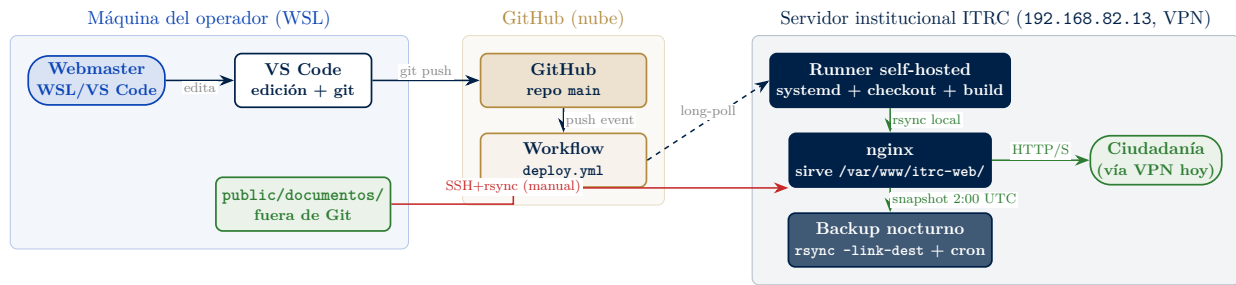
## Estado del despliegue (mayo 2026)

El portal está operativo en un **servidor de pruebas** accesible únicamente desde la VPN institucional (FortiClient) en `http://192.168.82.13/`. El despliegue automatizado funciona extremo a extremo. Pendiente para producción pública: dominio definitivo, certificado TLS, y resolución de la pieza CMS (ver §3.2).

## 1.1 Componentes principales

| Componente                             | Rol                                                                                                                                                          |
|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Astro 4.x</b>                       | Generador de sitio estático. Transforma archivos JSON/MD en HTML.                                                                                            |
| <b>GitHub</b>                          | Almacenamiento del código, control de versiones, y orquestador de despliegues.                                                                               |
| <b>GitHub Actions</b>                  | Pipeline CI/CD que construye y publica el sitio. Workflow: <code>.github/workflows/deploy.yml</code> .                                                       |
| <b>Runner self-hosted</b>              | Servicio <code>systemd</code> ejecutado por el usuario <code>github-runner</code> dentro del servidor. Etiquetas: <code>[self-hosted, itrc-server]</code> .  |
| <b>Ubuntu + nginx<br/>rsync + cron</b> | Servidor institucional que sirve los archivos al usuario final. Política de respaldos local con snapshots tipo <i>Time Machine</i> usando <i>hardlinks</i> . |
| <b>Sveltia CMS</b>                     | Panel visual de edición (en evaluación; ver §3.2).                                                                                                           |

## 2 Diagrama de arquitectura



### Cómo leer el diagrama

Las flechas **azules continuas** representan acciones del usuario o del sistema disparadas por un **push**. Las **azules punteadas** representan el *long-poll* por el cual el runner espera trabajos de GitHub (sin abrir puertos de entrada). Las **verdes** representan el camino del contenido publicado hasta la ciudadanía y el respaldo nocturno. La flecha **roja** representa el canal manual de subida de binarios (PDFs, multimedia) que viven fuera del repositorio.

### Diferencia clave con la versión anterior

En la versión 1.0 de este documento el GitHub Action corría en la nube y se conectaba al servidor por SSH. En la versión actual, el **runner** vive **dentro del servidor** y se conecta a GitHub mediante *long-polling* HTTPS saliente. Esto elimina la necesidad de exponer SSH a Internet, eliminar llaves remotas, y simplifica la configuración del firewall.

### 3 Flujo de publicación paso a paso

El portal soporta cuatro flujos de publicación. Los dos primeros (*push* desde VS Code y CMS) actualizan el contenido editorial; los otros dos (binarios y *deploy* manual) son canales secundarios para casos específicos.

#### 3.1 Escenario 1: Webmaster edita con VS Code (flujo principal hoy)

Aplica para todos los cambios actuales: contenido editorial, código, configuración. Es el único flujo automatizado en operación a la fecha.

1. El webmaster ejecuta `git pull` en su copia local para sincronizarse.
2. Abre el proyecto en VS Code y modifica archivos: JSON de contenido en `src/content/`, código del portal, etc.
3. Ejecuta `npm run dev` localmente para previsualizar cambios en `http://localhost:4321`.
4. Satisfecho con el resultado, ejecuta `git commit` y `git push`.
5. GitHub recibe el push y, salvo que los archivos modificados estén en `paths-ignore` (`docs/**`, `README.md`, `CLAUDE.md`, `LICENSE`, `.gitignore`, `GEMINI.md`), dispara el workflow `deploy.yml`.
6. El *runner self-hosted* (que ya está esperando trabajos vía *long-poll*) ejecuta los pasos: `checkout`, `npm ci`, `npm run build` (con `SITE_URL` y `BASE_PATH` desde *repo variables*), y `rsync -a -delete -exclude=/documentos/` desde `dist/` a `/var/www/itrc-web/`.
7. El runner emite `sudo /usr/bin/systemctl reload nginx` (única regla `sudoers` que tiene). nginx recarga la configuración sin downtime.
8. Tiempo total: 1 a 3 minutos. El estado del workflow se observa en GitHub → Actions.

#### Configuración del workflow

El archivo `.github/workflows/deploy.yml` usa `concurrency: deploy-${{ github.ref }}` para serializar despliegues, `timeout-minutes: 15` como salvaguarda, y un bloque `env:` para todas las interpolaciones de `${{ ... }}` dentro de pasos `run:` (mitigación de *shell injection*, identificada por *semgrep*).

#### 3.2 Escenario 2: Webmaster edita con Sveltia CMS (*pendiente*)

Aplica para el flujo cotidiano que tendría el webmaster no técnico: publicar noticias, actualizar documentos, cambiar banners, modificar textos.

#### Estado actual: bloqueado

La ruta `/admin/` está deshabilitada en nginx hasta resolver la pieza CMS. Sveltia CMS, en su versión actual (0.128.x), sólo permite subir media a **servicios pagados** (Cloudinary, Uploadcare). Como el ITRC no cuenta con presupuesto para servicios externos y el portal maneja ~3.5 GB de PDFs, se requiere o bien (a) parchar Sveltia para que suba a un *endpoint* propio, (b) implementar un endpoint Node de subida y servirlo paralelo, o (c) cambiar a otro CMS Git-friendly (TinaCMS, Decap, Directus, Strapi, Payload). Decisión pendiente para sesión dedicada.

Cuando se resuelva, el flujo será:

1. El webmaster navega a `<dominio>/admin/` en su navegador.
2. Hace clic en *Login with GitHub*. El flujo OAuth lo lleva a autenticarse con su cuenta.
3. Una vez autenticado, ve el panel del CMS con las colecciones (noticias, páginas, slider, etc.).
4. Realiza los cambios usando el editor visual.
5. Hace clic en *Publish*. El CMS commitea al repo con autoría de la cuenta GitHub del editor.
6. A partir de ahí, mismo flujo que el Escenario 1 (push → runner → rsync → nginx).

### 3.3 Escenario 3: Subida de binarios al servidor

Los archivos binarios (PDFs, imágenes pesadas, multimedia) viven **fuera del repositorio Git**, sólo en el servidor (decisión de mayo 2026, ver §6.1). Para subirlos:

1. El operador edita o añade archivos localmente en `public/documentos/`.
2. Ejecuta `npm run deploy:binarios`, que invoca `ops/deploy-binarios.sh`.
3. El script abre una conexión SSH (usuario `admweb`, autenticado con llave) hacia `192.168.82.13` y sincroniza con `rsync -avz -no-o -no-g -no-p -no-t` (sin `-delete` por seguridad).
4. Los binarios quedan disponibles en `/var/www/itrc-web/documentos/` para servirse con cache largo (`expires 7d`).

#### Por qué un canal manual y no el workflow

El runner haría `rsync -delete` sobre el webroot, borrando todo lo que no esté en el repo. Como los binarios no están en el repo, los borraría. La regla `-exclude=/documentos/` en el workflow protege el directorio durante despliegues automáticos y obliga a manejar binarios por un canal separado y consciente.

### 3.4 Escenario 4: *Deploy* manual de fallback

Si el runner está caído (mantenimiento, fallo del servicio), el operador puede desplegar manualmente desde su máquina:

1. `npm run build` localmente.
2. `npm run deploy` (script `ops/deploy.sh`, lee `.env.deploy` con `SSH_BIN`, `DEPLOY_HOST`, `DEPLOY_USER`, `DEPLOY_PATH`).
3. El script hace `rsync` desde `dist/` hacia el servidor con las mismas exclusiones.

Este flujo no requiere que el runner esté corriendo y funciona sólo con SSH desde la máquina del operador.

#### Trazabilidad

Cada commit queda registrado en el historial Git con autor, fecha y descripción. Cada despliegue automático queda registrado en GitHub → Actions con SHA, autor y mensaje del commit. El *deploy* manual queda en el historial `rsync` del operador, sin trazabilidad central — por eso es un escenario de excepción, no rutinario.

## 4 Configuración del servidor (Ubuntu + nginx)

### 4.1 Servidor de pruebas actual

| Atributo          | Valor                                                            |
|-------------------|------------------------------------------------------------------|
| Hostname / IP     | 192.168.82.13 (sólo accesible vía VPN FortiClient institucional) |
| Sistema operativo | Ubuntu Server 22.04 LTS                                          |
| nginx             | 1.18+ (paquete oficial Ubuntu)                                   |
| Espacio en disco  | 16 GB total, ~12 GB libres tras instalación + 2 snapshots        |
| RAM               | 2 GB                                                             |
| CPU               | 1 vCPU (suficiente para servir estáticos)                        |
| Puertos abiertos  | 80 (HTTP, vía VPN), 22 (SSH)                                     |
| TLS               | <b>pendiente</b> (sin dominio público todavía)                   |

### 4.2 Cuentas y permisos del servidor

El servidor cuenta con dos usuarios funcionales y un grupo `deploy` compartido con `setgid` para que ambos puedan escribir el mismo árbol de archivos:

| Usuario                     | Rol                                                                                                                                                                                                                                                        |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>admweb</code>         | Operador humano. Acceso SSH con llave; recibe binarios desde la máquina local del operador ( <code>npm run deploy:binarios</code> ). Miembro del grupo <code>deploy</code> .                                                                               |
| <code>github-runner</code>  | Cuenta de servicio que ejecuta el runner self-hosted (vía <code>systemd</code> ). Tiene una regla <code>sudoers</code> muy acotada que le permite <i>únicamente</i> <code>/usr/bin/systemctl reload nginx</code> . Miembro del grupo <code>deploy</code> . |
| <code>deploy</code> (grupo) | Grupo compartido. Webroot <code>/var/www/itrc-web/</code> es del grupo <code>deploy</code> con <code>g+rwX</code> y bit <code>g+s</code> ( <code>setgid</code> ) para que archivos nuevos hereden el grupo.                                                |

#### Sudoers del runner

```
github-runner ALL=(root) NOPASSWD: /usr/bin/systemctl reload nginx
```

Nada más. El runner no puede instalar paquetes, cambiar configuración, ni reiniciar servicios. Esto limita el blast radius de un compromiso del repo o del workflow.

### 4.3 Estructura del sitio en el servidor

```
/var/www/itrc-web/      <-- webroot (raíz para nginx)
|- index.html           <-- copiados aquí por el runner
|- _astro/              <-- assets contruidos por Astro
|- ...                  <-- resto del sitio compilado
\-- documentos/         <-- binarios (PDFs, multimedia)
                        NO los toca el runner;
                        se suben por canal separado
```

### 4.4 Configuración nginx mínima (estado actual)

Archivo `/etc/nginx/sites-available/itrc-web`:

```
server {
    listen 80 default_server;
    server_name _;

    root /var/www/itrc-web;
    index index.html;

    # Compresión
    gzip on;
    gzip_types text/plain text/css application/javascript application/json
        image/svg+xml;

    # Cache largo para assets contruidos
    location /_astro/ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
    location /documentos/ {
        expires 7d;
        add_header Cache-Control "public";
    }

    # Bloqueo temporal del CMS hasta resolver evaluación
    location /admin/ {
        return 503;
    }

    # Routing Astro
    location / {
        try_files $uri $uri.html $uri/index.html =404;
    }
}
```

### TLS y dominio: pendiente

Cuando se asigne un dominio público y se autorice la salida de la VPN, se añade un **server** de *listen 443* con *Let's Encrypt*, redirección 80 → 443, y los *headers* HSTS / X-Frame-Options / Referrer-Policy. Hoy todo eso está documentado en `docs/manual-operador/` pero sin aplicar al ser un servidor exclusivamente VPN-interno.



## 5 Autenticación y administración de usuarios

### Aplicabilidad

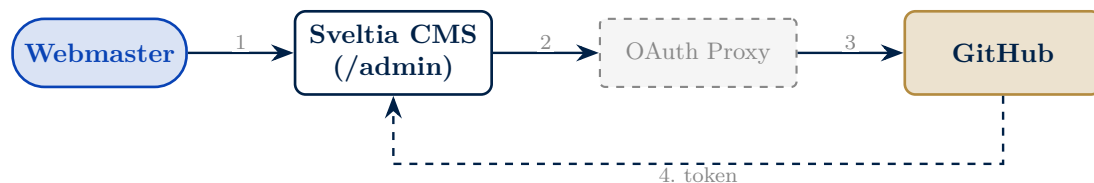
Este modelo aplica para los flujos basados en Git (Escenarios 1 y 2). Hoy se usa sólo el Escenario 1 (push desde VS Code), por lo que la administración se reduce a colaboradores del repositorio. Cuando se reactive el Escenario 2 (CMS), entra en juego el flujo OAuth descrito abajo.

### 5.1 Modelo de cuentas

Cada webmaster de la institución cuenta con una **cuenta personal de GitHub**. No se comparten credenciales entre personas. Las razones son:

- **Trazabilidad:** todos los commits en el historial quedan asociados a un nombre real.
- **Seguridad:** si una cuenta se compromete, se revoca su acceso individualmente sin afectar al resto.
- **Escalabilidad:** añadir o retirar personas es cuestión de un clic, sin pasar por el equipo técnico cada vez.
- **Auditoría:** cualquier auditoría posterior puede reconstruir quién hizo qué con precisión.

### 5.2 Flujo de autenticación (OAuth con GitHub) — *pendiente de despliegue*



1. El webmaster entra al CMS y solicita login.
2. El CMS redirige al OAuth Proxy (aloja credenciales de la app OAuth registrada en GitHub).
3. El Proxy redirige a GitHub para que el usuario autorice.
4. GitHub entrega un token al CMS via el Proxy. El CMS queda autenticado durante la sesión (típicamente 8 horas).

### 5.3 Alta de un webmaster nuevo

El procedimiento, a cargo del coordinador técnico:

1. El webmaster crea (o ya tiene) una cuenta personal en [github.com](https://github.com). Se recomienda usar el correo institucional.
2. El coordinador accede al repositorio del portal en GitHub, en la sección *Settings* → *Collaborators*, y añade al nuevo webmaster con el rol **Write**.
3. GitHub envía una invitación por correo. El webmaster la acepta.
4. El webmaster clona el repositorio localmente, instala dependencias (`npm install`) y prueba `npm run dev`. Si el sitio carga, está listo para empujar cambios.
5. Cuando se reactive el CMS, también probará entrar a `<dominio>/admin/` y hacer login OAuth.

## 5.4 Roles sugeridos

| Rol GitHub      | Puede hacer                       | Para quién                  |
|-----------------|-----------------------------------|-----------------------------|
| <i>Admin</i>    | Todo + gestionar colaboradores    | 1–2 coordinadores técnicos  |
| <i>Maintain</i> | Push + gestionar issues           | Coordinadores de contenido  |
| <i>Write</i>    | Push a main (suficiente para CMS) | Webmasters regulares        |
| <i>Read</i>     | Solo lectura                      | Auditoría, visores externos |

## 5.5 Baja de un webmaster

Al retirar a una persona:

- Se remueve su acceso en *Settings* → *Collaborators* → *Remove*.
- Los commits previos quedan en el historial (no se eliminan, por auditoría).
- No se revoca la aplicación OAuth: está asociada al portal, no a la persona.

## 5.6 Buenas prácticas de seguridad

- Exigir **autenticación en dos pasos (2FA)** en todas las cuentas que tengan acceso al repo.
- Revisar trimestralmente la lista de colaboradores y retirar accesos inactivos.
- El *Client Secret* del OAuth App nunca debe aparecer en código ni en commits.
- Ante sospecha de cuenta comprometida: remover acceso inmediato, cambio de contraseña y rotación de 2FA por parte del usuario, y reincorporación controlada.

## 6 Consideraciones operativas

### 6.1 Manejo de binarios

#### Pivote de mayo 2026

La versión 1.0 de este documento describía un modelo donde los binarios vivían dentro del repositorio. Ese enfoque se descartó tras un incidente de pérdida de 3.5 GB durante un despliegue: el `rsync -delete` del workflow borró el directorio de binarios cuando el runner clonó el repo limpio (sin esos archivos). El modelo actual los mantiene fuera del repositorio.

Los archivos binarios del portal (PDFs, hojas de cálculo, imágenes pesadas, audios, videos) residen **únicamente en el servidor**, en el directorio `/var/www/itrc-web/documentos/`. No se versionan en Git. Esto implica:

- El repositorio Git contiene únicamente el código del sitio y el contenido editorial (JSON / Markdown), con un tamaño cómodo (~50 MB) que clona en segundos.
- Los binarios se suben por canal manual (`npm run deploy:binarios`, ver Escenario 3).
- El workflow del runner usa `-exclude=/documentos/` para no tocarlos durante despliegues, evitando una repetición del incidente.
- La ruta de servido en el código sigue siendo `/documentos/` a través de `DOCS_BASE_URL`.
- La fuente de verdad de los binarios es el servidor + el snapshot nocturno (ver §6.4). Cuando el operador necesita una copia, hace `rsync` desde el servidor a su máquina local.

#### Implicación: el CMS necesita un endpoint de uploads

Como los binarios no están en Git, el CMS visual (cuando se reactive) no puede subirlos vía `commit API`. Necesita un endpoint HTTP propio que reciba el archivo, lo deposite en `/var/www/itrc-web/documentos/` y devuelva la URL pública. Esa pieza está pendiente y depende de la decisión de CMS (§3.2).

### 6.2 Rollback (revertir un cambio)

**Para contenido o código (en Git):**

1. Identificar el commit problemático con `git log -oneline`.
2. Ejecutar `git revert <hash>` — crea un nuevo commit que deshace los cambios.
3. `git push`. El runner despliega la versión restaurada en 1–3 minutos.

**Para binarios (en servidor):**

1. Identificar el snapshot que tiene la versión correcta: `ls /var/backups/itrc-web/daily/`.
2. Restaurar selectivamente: `sudo cp /var/backups/itrc-web/daily/daily.N/documentos/<archivo>/var/v`
3. Si se requiere restaurar todo el directorio, usar `rsync -a` desde el snapshot (ver docs/BACKUP.md).

#### Nunca forzar el historial

**No usar `git push -force`** sobre la rama principal. El historial es auditable y debe preservarse. La forma correcta de deshacer cambios publicados es siempre con `git revert`, que deja evidencia de la corrección en el log.

### 6.3 Monitoreo y logs

- **Logs de nginx:** `/var/log/nginx/access.log` y `error.log`. Rotación semanal estándar.
- **Estado de despliegues:** pestaña *Actions* del repositorio en GitHub. Cada workflow se registra con SHA, autor, mensaje, duración y éxito/fallo.
- **Estado del runner:** `sudo systemctl status actions.runner.<repo>.<host>.service`. El runner debe aparecer como *active (running)* y conectado al repo.
- **Log del backup:** `/var/log/itrc-backup.log`. Buscar "FIN BACKUP" para confirmar éxito de la última corrida.
- **Disponibilidad:** monitoreo externo opcional (uptime pings) cuando exista dominio público.

### 6.4 Política de respaldos local

El servidor ejecuta un cron nocturno (2:00 UTC) que invoca `/usr/local/bin/itrc-backup.sh` (commiteado en el repo como `ops/server-backup.sh`). La estrategia es *Time Machine* basada en `rsync -link-dest`: cada snapshot se ve como árbol completo navegable, pero archivos sin cambios son *hardlinks* al snapshot anterior, así que el costo en disco es  $\approx$  tamaño actual + suma de deltas diarios.

| Tipo              | Cuántos | Frecuencia                                                                |
|-------------------|---------|---------------------------------------------------------------------------|
| Daily             | 7       | Cada noche, rotación FIFO                                                 |
| Weekly            | 4       | Cada domingo (clonado de <code>daily.0</code> con <i>hardlinks</i> )      |
| Monthly           | 6       | Día 1 de cada mes (clonado de <code>daily.0</code> con <i>hardlinks</i> ) |
| Configs (tarball) | 30      | Uno por día, incluye nginx, sudoers, ufw, lista de paquetes, kernel       |

Espacio físico estimado para volumen actual ( $\sim 3.7$  GB de webroot): 6–8 GB para todos los snapshots combinados. Con 12 GB libres en disco y umbral de alerta a 85 %, hay margen.

#### Limitaciones conocidas

El backup es **local al mismo servidor**. Si el servidor entero falla (disco principal corrupto, VM perdida), los respaldos se pierden con él. Mitigación pendiente: cuando se asigne una segunda máquina institucional, configurar *push* periódico de los snapshots y los *tarballs* de configs a esa máquina (o a un repo Git privado para los configs, que son livianos).  
Tampoco hay **notificación de fallo** hoy: el cron tiene `MAILTO=daniel@digitalia.gov.co` pero `postfix` no está instalado. Mientras: revisar `/var/log/itrc-backup.log` periódicamente.

Detalle completo de la política, estructura en disco y procedimientos de restauración en `docs/BACKUP.md`.

### 6.5 Respaldo del repositorio (Git)

Para el lado Git, el modelo provee respaldo implícito:

- El repositorio en GitHub es un backup por sí mismo (replicado en infraestructura GitHub).
- Cada webmaster con su clon local tiene otra copia completa.
- El historial Git permite restaurar cualquier versión anterior por fecha o commit.

## Referencias

---

### Documentos internos del repositorio (fuente de verdad operativa):

- docs/DEPLOY.md — Detalle de los tres flujos de despliegue (auto, fallback, binarios) y diagnóstico de fallos.
- docs/BACKUP.md — Política de respaldos, estructura en disco y procedimientos de restauración.
- docs/manual-operador/ — Manual del operador (instalación, configuración, mantenimiento del servidor).
- ops/deploy.sh, ops/deploy-binarios.sh, ops/server-backup.sh — Scripts ejecutables.
- .github/workflows/deploy.yml — Workflow del runner self-hosted.
- REFERENCIAS-FALTANTES.md — Auditoría de enlaces y omisiones detectadas durante la migración.

### Documentación externa:

- Astro: <https://astro.build>
- Sveltia CMS: <https://github.com/sveltia/sveltia-cms>
- GitHub Actions self-hosted runners: <https://docs.github.com/actions/hosting-your-own-runners>
- nginx documentation: <https://nginx.org/en/docs/>
- Let's Encrypt / Certbot: <https://certbot.eff.org>
- rsync *link-dest* (estrategia Time Machine): <https://rsync.samba.org/documentation.html>